Secrets & keys — where each one goes (both products)

SECTION 06-operations-it TYPE document STATUS active OWNER Founder UPDATED 2026-08-19

One index for **where** every secret, key and price is configured across the two backends. **No secret values live here** — only names and locations. The step-by-step is in the authoritative runbooks (linked at the end); this page is the map on top of them.

Golden rules

- **Never** put a secret value in git, in chat, or in code. Secrets live only in **Google Secret Manager**; the service reads them at runtime as mounted env vars.
- `VITE_*` variables are **public build-time** values shipped to the browser — **not secrets** (publishable keys, site URLs, analytics ids). Safe to expose.
- The IBAN never leaves this repo (see `01-governance/company-facts.md`).
- Neither product uses Firebase. There is no `firebase functions:secrets:set` step any more, and no `functions/.env.*` file — everything is a Cloud Run revision setting. See `hosting-facts.md`.

A) flygaca.com — `ay2m/FlyGACA` · Cloud Run service `flygaca-api` · region `me-central2` (Dammam)

Secrets → Google Secret Manager, one secret per value:

```
printf '%s' 'VALUE' | gcloud secrets create NAME --data-file=-
```

then mounted onto the revision with `gcloud run deploy --set-secrets=ENV_VAR=secret-name:latest`. Grant the Cloud Run service account `roles/secretmanager.secretAccessor` (and `roles/cloudsql.client` for the database).

Env var	Secret Manager name	What / where to get it	Required
`DATABASE_URL`	`database-url`	Cloud SQL unix-socket URL — `postgresql://…@/flygaca?host=/cloudsql/PROJECT:REGION:INSTANCE`	✅ boot
`SESSION_SECRET`	`session-secret`	You generate (`openssl rand -base64 48`) — signs the session JWT; **min 32 chars**	✅ boot

Env var	Secret Manager name	What / where to get it	Required
`GOOGLE_OAUTH_CLIENT_SECRET`	`google-oauth-secret`	GCP → APIs & Services → Credentials → Web application client	✅ Google sign-in
`GOOGLE_GENAI_API_KEY`	`genai-api-key`	Gemini API key — Google AI Studio (aistudio.google.com)	✅ chat
`MOYASAR_SECRET_KEY`	`moyasar-secret-key`	Moyasar dashboard → Developers → API keys (`sk_live_…`)	✅ billing
`MOYASAR_WEBHOOK_SECRET`	`moyasar-webhook`	Moyasar webhook shared secret (signature is verified over the raw body)	○ backstop
`MAIL_API_KEY`	`mail-api-key`	Resend-compatible transactional-mail key. Absent ⇒ mails are logged, not sent	○
`CRON_SECRET`	`cron-secret`	You generate (`openssl rand -hex 32`) — guards `POST /api/billing/renew`	✅ renewals

`assertRequiredConfig()` runs before the listener binds, so a revision missing `DATABASE_URL` — or carrying a too-short `SESSION_SECRET` — fails its health check instead of 500-ing later.

Params (not secrets) → `gcloud run deploy --set-env-vars=…`. `server/src/config.ts` is the single place the server reads `process.env`:

- Origins (required): `APP_ORIGIN=https://flygaca.com`, `API_ORIGIN=https://api.flygaca.com` — `API_ORIGIN` must match the OAuth redirect URI exactly or the callback fails with `redirect_uri_mismatch`. `APP_ORIGIN` also builds the Moyasar callback URL.
- Also required-ish: `NODE_ENV=production`, `GOOGLE_OAUTH_CLIENT_ID`, `MAIL_FROM`.
- **Prices** (SAR integers, **no defaults — a missing one throws at checkout**, it does *not* silently charge SAR 0): `PRICE_PRO_MONTHLY`, `PRICE_PRO_ANNUAL`, `PRICE_PASS`, `PRICE_CREDITS`, `PRICE_PREP_PACK_ESSENTIAL`, `PRICE_PREP_PACK_STANDARD`, `PRICE_PREP_PACK_COMPLETE`, `PRICE_BUNDLE`, `PRICE_COHORT`. (`PRICE_PREP_PACK` also exists as the generic single-pack price.)
- Optional tuning: `CHAT_FREE_DAILY_LIMIT`, `ANON_DAILY_LIMIT`, `CHAT_CREDIT_PACK_SIZE`, `CHAT_ENABLED`, `RETRIEVE_K`, `REFUSE_SCORE`, `GROUNDED_SCORE`, `CORPUS_URL`, `SESSION_TTL_DAYS`, `SESSION_COOKIE_DOMAIN`, `DATABASE_POOL_MAX`, `EXTRA_ALLOWED_ORIGINS`.

Public (not secrets) → root `.env.local`, baked into the SPA build as `VITE_*`: `VITE_API_BASE_URL` (or `VITE_API_SAME_ORIGIN=1` when the load balancer serves both from one origin), `VITE_MOYASAR_PUBLISHABLE_KEY` (`pk_live_…`), plus the optional `VITE_DATA_BASE_URL`, `VITE_SITE_URL`, `VITE_GA_MEASUREMENT_ID`.

> With no `VITE_API_BASE_URL` the SPA ignores the API entirely and runs local-first — corpus, tools, study and logbook all work; accounts/sync/billing stay dark. That is the default for CI and the preview build, and it is why no CI job needs production secrets.

Deploy / CI credentials (repo settings → Secrets): a GCP service-account credential for `gcloud run deploy` + the bucket sync (`GCP_SA_KEY` / Workload Identity), and `CLOUDFLARE_API_TOKEN` · `CLOUDFLARE_ACCOUNT_ID` for the Worker mirror. *(There is no `FIREBASE_SERVICE_ACCOUNT` — that credential belonged to the archived predecessor repo.)*

B) captadel.com — `ay2m/Captain-Adel` · Cloud Run · region `me-central2` (Dammam)

Secrets → Google Secret Manager, via `printf '%s' "VALUE" | gcloud secrets create NAME --data-file=-`, or the batch shortcut `export NAME=… … && ./deploy/deploy.sh secrets`:

Secret	What / where to get it	Required
`GEMINI_API_KEY`	Gemini API key — Google AI Studio (same key as flygaca, different name)	✅
`MOYASAR_SECRET_KEY` · `MOYASAR_PUBLISHABLE_KEY` · `MOYASAR_WEBHOOK_SECRET`	Moyasar dashboard	✅ billing
`MOYASAR_PRICE_MONTHLY_SAR` (35) · `MOYASAR_PRICE_ANNUAL_SAR` (299)	you set	✅
`CRON_SECRET`	you generate (`openssl rand -hex 32`) — guards the renewals route	✅ renewals
`ADEL_API_KEY`	a shared secret you invent (trusted-tier API)	○
`ALLAM_*`, `EMBEDDINGS_*`, `RERANK_*`	in-Kingdom Arabic provider / dense retrieval	○ later

Non-secret env → `--set-env-vars` in `deploy.sh`: `SITE_URL=https://captadel.com`, `MODEL_PROVIDER`, `ARABIC_PROVIDER`, `ADEL_LAUNCH_MODE`, `ADEL_DAILY_*`.

> `deploy.sh` still offers `REGION=me-central1` as a fallback if Dammam rejects the deploy. **Do not take it for production.** `me-central1` is Doha, Qatar — outside the Kingdom — and captadel handles real user queries, which are personal data under PDPL. If `me-central2` is unavailable, escalate rather than deploy west.

GitHub Actions: `GCP_SA_KEY`.

Five things to watch

1. **One Gemini key, two names** — `GOOGLE_GENAI_API_KEY` (flygaca) vs `GEMINI_API_KEY` (captadel). Same value from Google AI Studio, set in each product's own store.
2. **Prices are named differently in each product.** flygaca uses bare `PRICE_*` env vars (plain SAR integers) on the Cloud Run revision; captadel still uses `MOYASAR_PRICE_*_SAR`. The old flygaca `MOYASAR_PRICE_*_SAR` names — and every `*_STUDENT_*` price key — no longer exist; if you find them in a doc or a script, they are stale.

3. **One Moyasar account, two secret stores + two webhooks** — set the Moyasar keys in both stores, and register **two** webhook endpoints in the Moyasar dashboard: `https://api.flygaca.com/api/billing/webhook/moyasar` and `https://captadel.com/v1/billing/webhook`. On flygaca the webhook is defence in depth — the browser's return leg calls `POST /api/billing/confirm`, which fetches the payment server-to-server; both funnel into the same idempotent `fulfil()`.
4. **Publishable key** — public build var on flygaca (`VITE_MOYASAR_PUBLISHABLE_KEY`), but a stored secret on captadel (`MOYASAR_PUBLISHABLE_KEY`, served via `/v1/config`).
5. **No Stripe, no Firebase, no App Check.** Any "Stripe", `firebase functions:secrets:set`, `functions/.env.*`, `ENFORCE_APP_CHECK` or `ADEL_APPCHECK_MODE` mention in an older doc is stale — both products use Moyasar, and flygaca's secrets are plain Secret Manager entries mounted onto a Cloud Run revision.

Authoritative step-by-step

- flygaca.com: `ay2m/FlyGACA/docs/RUNBOOK-deploy.md` (project setup, Secret Manager entries, the `--set-secrets` / `--set-env-vars` deploy line, the Cloud Scheduler renewal job) and `ay2m/FlyGACA/docs/BILLING.md` (checkout → confirm → webhook → renewal).
- captadel.com: `ay2m/Captain-Adel/docs/RUNBOOK-captadel-saas.md` §3 (Moyasar keys, webhook, Apple Pay, renewals), `ay2m/Captain-Adel/docs/RUNBOOK-captadel-deploy.md` (Gemini + project + deploy).